Lecture Notes for

Neural Networks I

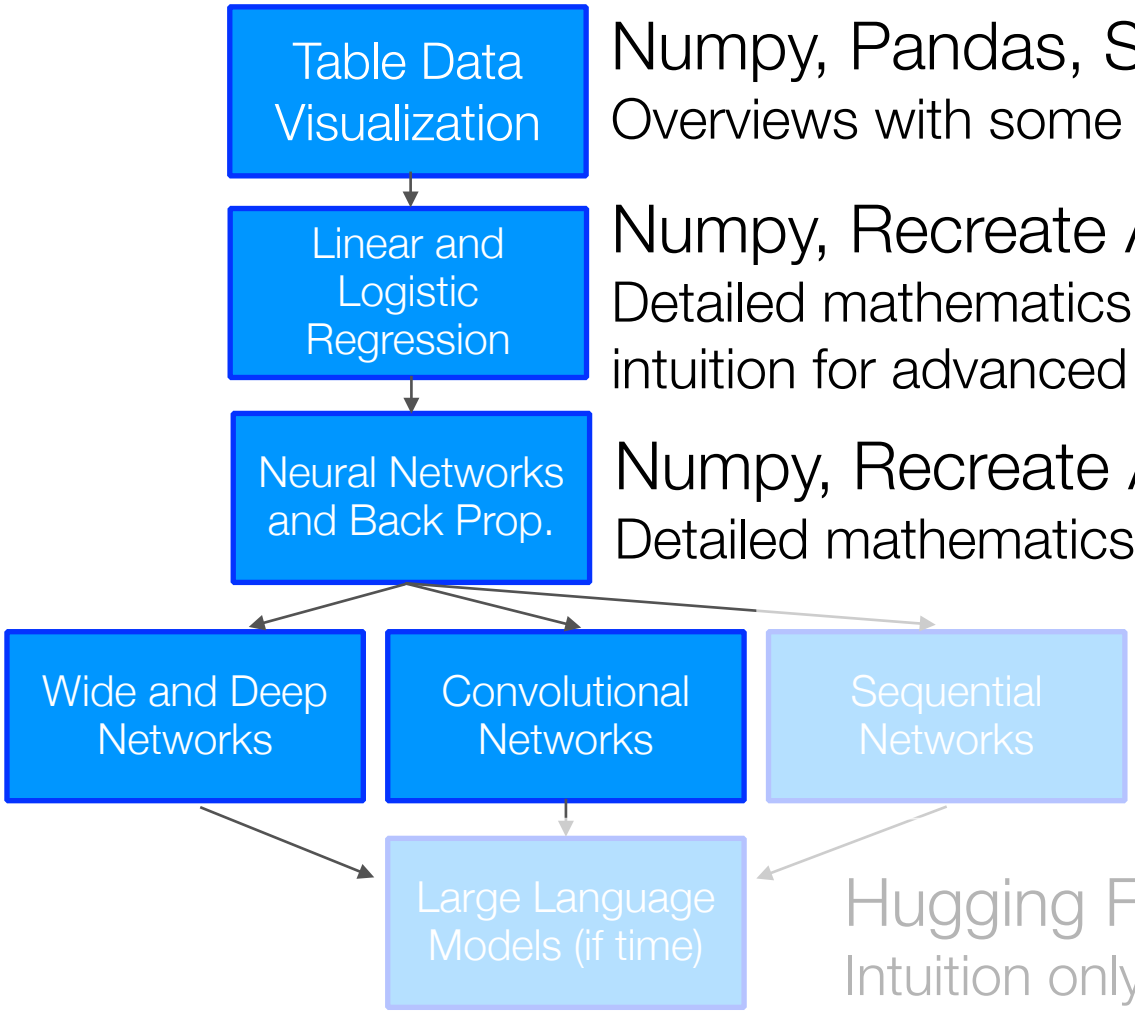


CNN Architectures

Class logistics and Agenda

- Logistics:
  - Next Week
  - Grading Update
- Agenda:
  - CNN Review
  - CNN Town Hall
  - More Advanced CNNs

Class Overview, by topic



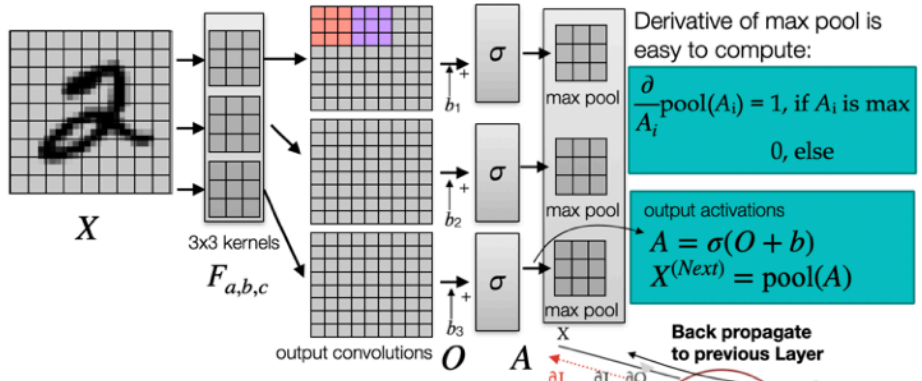
Numpy, Pandas, Seaborn
Overviews with some in-depth discussion

Numpy, Recreate API for Scikit-learn
Detailed mathematics for simple optimization
intuition for advanced optimization

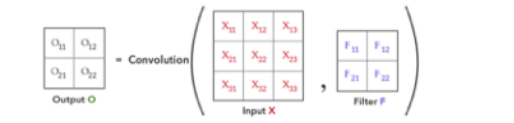
Numpy, Recreate API for DL Frameworks
Detailed mathematics for NN operations

Hugging Face
Intuition only

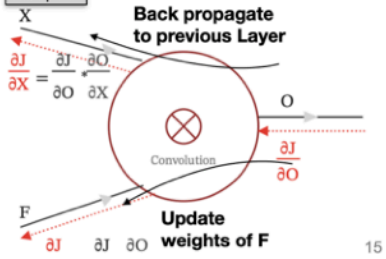
Last Time:



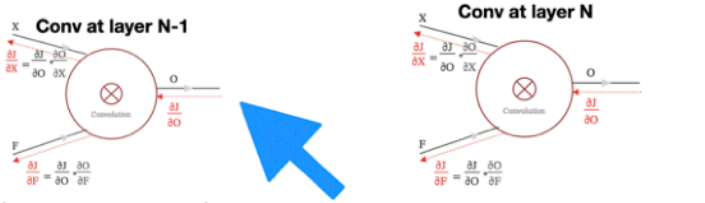
Derivative of convolution is more involved:

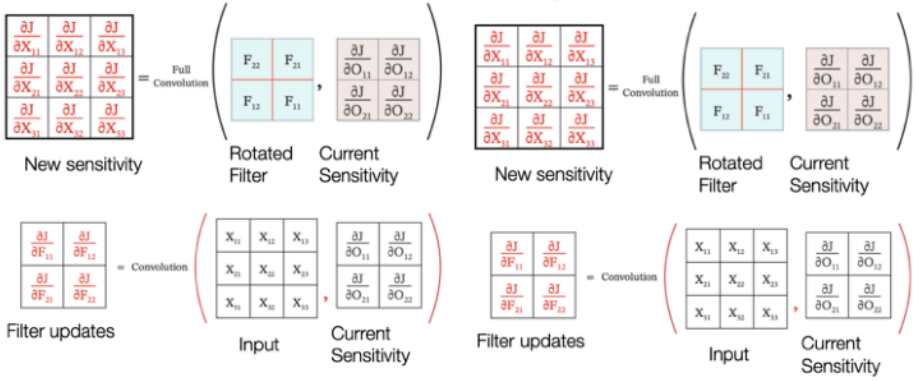


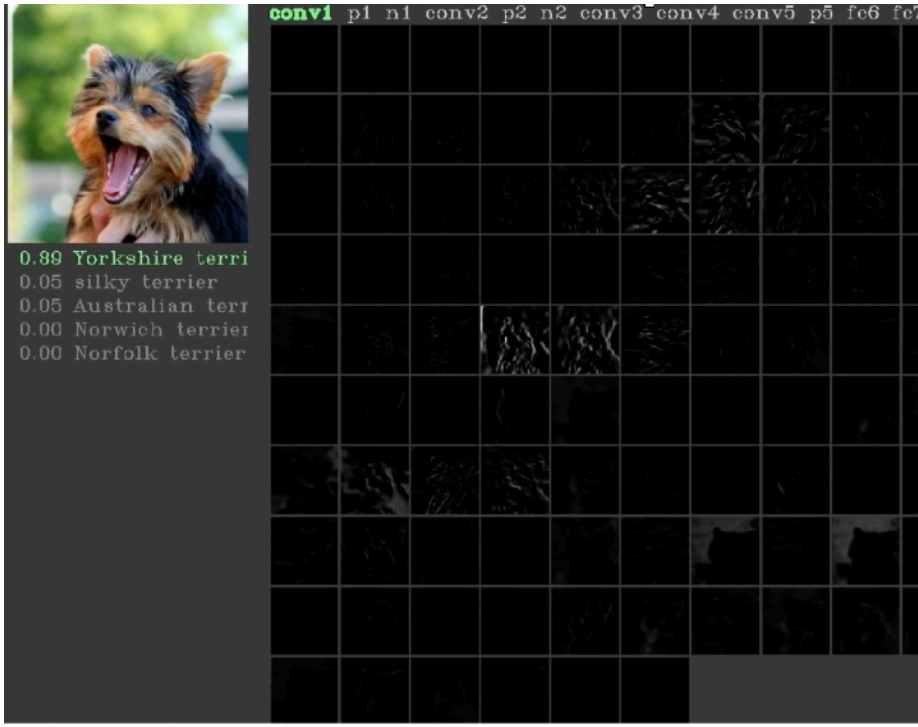
https://medium.com/@pavisj/convolutions-and-backpropagations-46026a8f5d2c



15



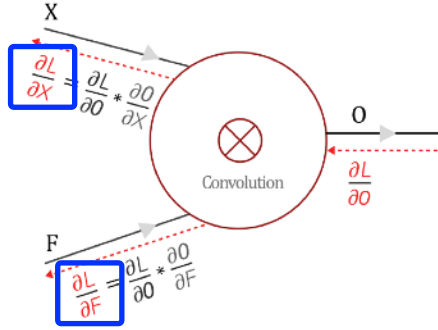




CNNs Back Propagation

Sensitivity to layer in back propagation

$$\frac{\partial J_{obj}}{\partial X^{(N)}} = V^{(N)} = \frac{\partial O^{(N)}}{\partial X^{(N)}} \cdot \frac{\partial A^{(N)}}{\partial O^{(N)}} \cdot V_{pool}^{(N+1)}$$



Output of Layer N-1

Input of Layer N

Filter Output of Layer N

Activations of N

Pooled Activations of Layer N (Output)

Input to Layer N+1

$X_{i,j,c}^{(N)}$

$\frac{\partial O^{(N)}}{\partial X^{(N)}}$

$O_{i,j,\hat{c}}^{(N)}$

Bias per Channel

$+ \; b_{\hat{c}}^{(N)}$

$\phi(O_{i,j,\hat{c}}^{(N)} + b_{\hat{c}}^{(N)})$

OR

$A_{i,j,\hat{c}}^{(N)}$

$X_{i,j,\hat{c}}^{(N+1)}$

Filtering

Activation

Pooling

$F_{a,b,c,\hat{c}}^{(N)}$

one filter in Layer N

$\frac{\partial O^{(N)}}{\partial F^{(N)}}$

$\frac{\partial A^{(N)}}{\partial O^{(N)}}$

$V_{pool}^{(N+1)}$

sensitivity through pooling

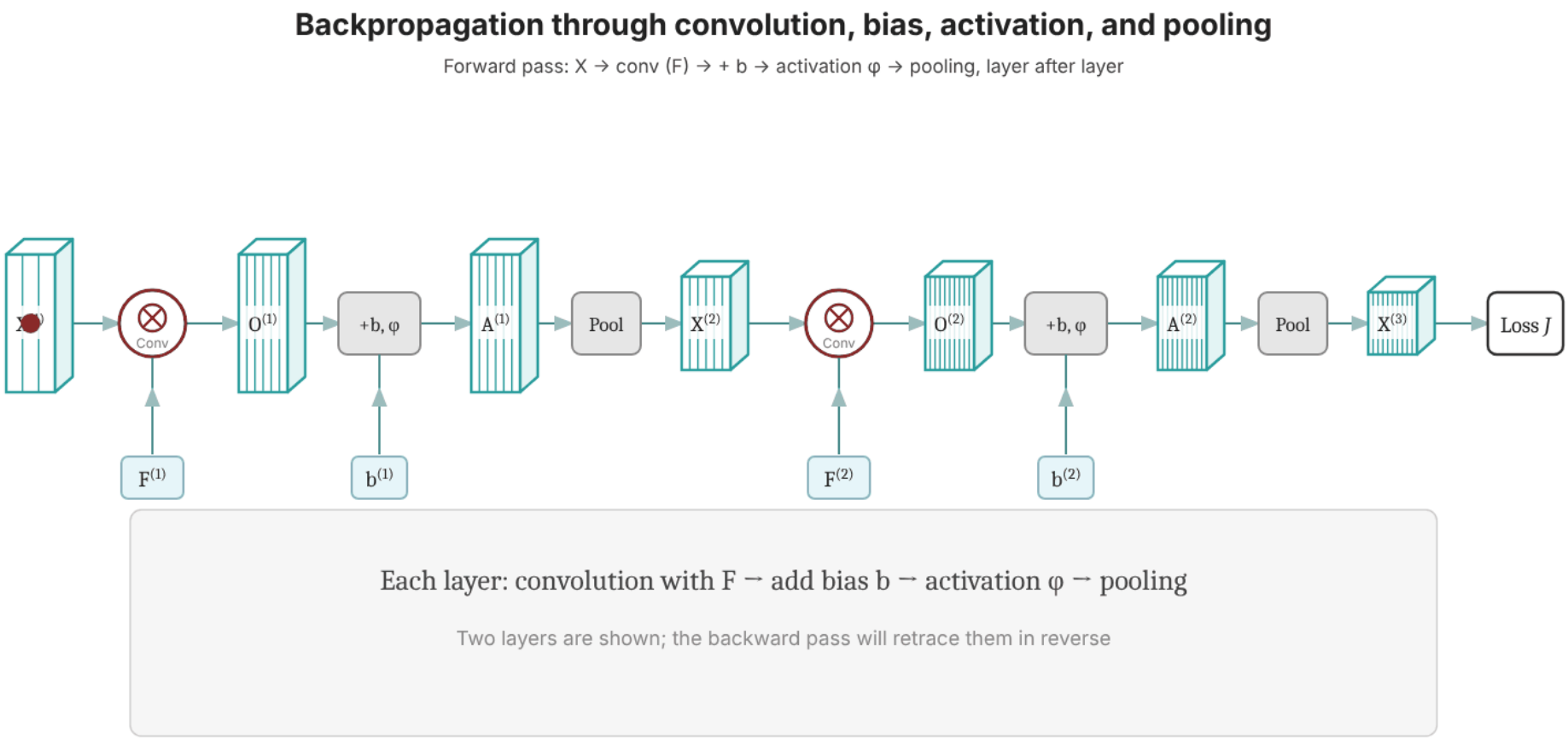
$V^{(N+1)}$ sensitivity $\frac{\partial J_{obj}}{\partial X^{(N+1)}}$

Just incorporate sensitivity, to get weight update

$$\frac{\partial J_{obj}}{\partial F^{(N)}} = \frac{\partial O^{(N)}}{\partial F^{(N)}} \cdot \frac{\partial A^{(N)}}{\partial O^{(N)}} \cdot V_{pool}^{(N+1)}$$

nnels x Rows x Columns

Summary


Backpropagation through convolution, bias, activation, and pooling
Forward pass: X → conv (F) → + b → activation φ → pooling, layer after layer
Conv
O(1)
+b, φ
A(1)
Pool
X(2)
Conv
O(2)
+b, φ
A(2)
Pool
X(3)
Loss J
F(1)
b(1)
F(2)
b(2)
Each layer: convolution with F → add bias b → activation φ → pooling
Two layers are shown; the backward pass will retrace them in reverse

Self-test

- For each traditional convolutional layer in a CNN, there are two convolutions per filter in the layer during back propagation.
  - **False**. There is a single convolution needed during back propagation for each filter.
  - **False**. The first convolutional layer has only one convolution needed, but all others have two per filter.
  - **True**. The filter weights are updated through two convolutions, one for each dimension.
  - **True**. One convolution is needed for pooling, and the other for filter weight updates.

Demo Finish, if needed…

Convolutional Neural Networks
in PyTorch

Demo



`09. Convolutional Neural Networks.ipynb`

Image Data Augmentation

```
import torchvision.transforms as transforms

trans = transforms.Compose([
            transforms.RandomCrop(64, padding=4),
            transforms.RandomHorizontalFlip(),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[… values …]),
        ])

full_train_dataset = datasets.ImageFolder(root='path', transform=trans)
```



RandomRotation( )

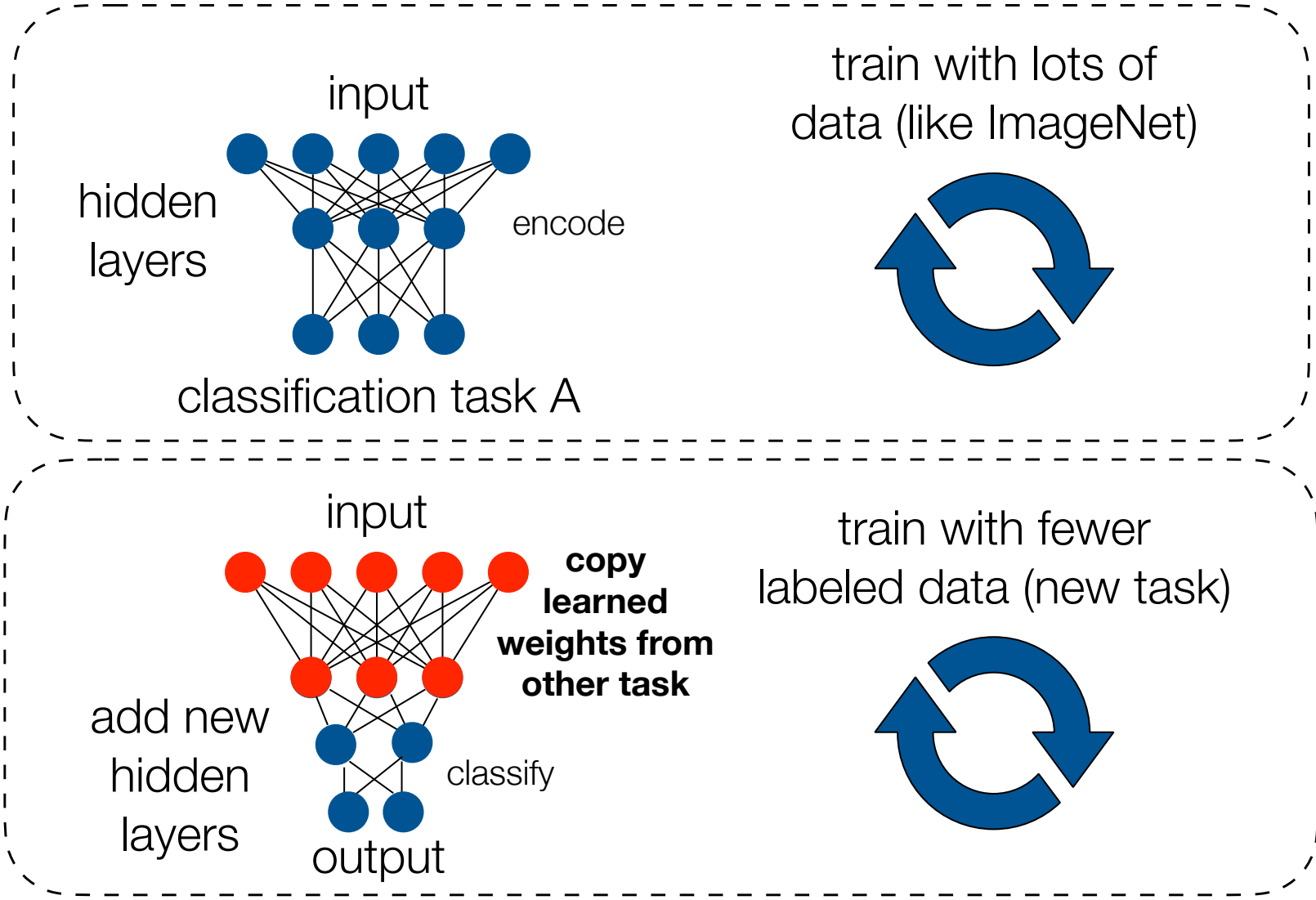




https://machinelearningmastery.com/how-to-configure-image-data-augmentation-when-training-deep-learning-neural-networks/

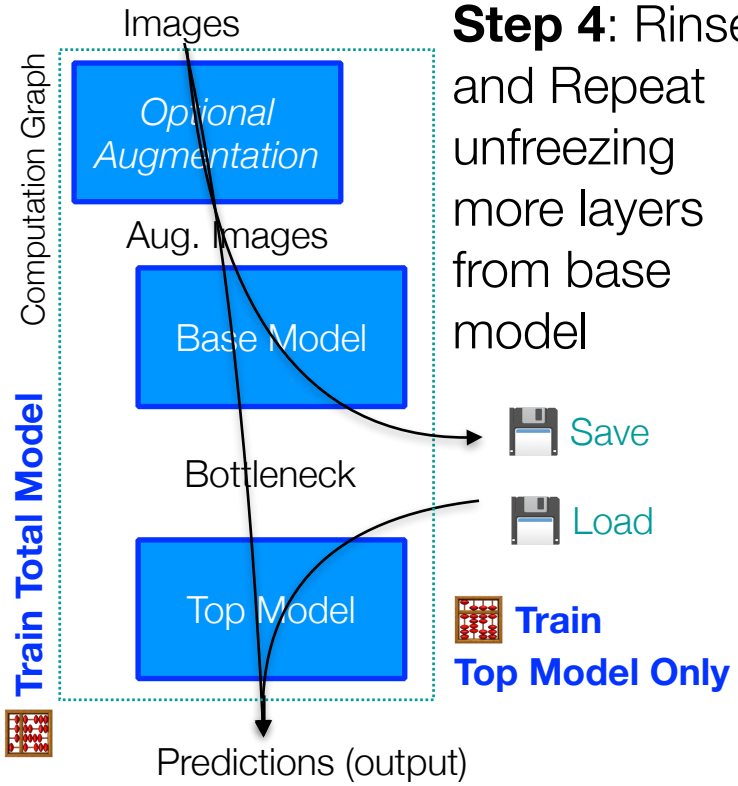
Transfer Learning

- transfer learning: a basic primer



Freezing and Fine-tuning Efficiently

- **Step 1:** Freeze entire base model:
  - No update during back-propagation
  - *Optional: Augment a set of training data*
  - Send training dataset through base model
    - Save out bottleneck features
- **Step 2:** Train bottleneck features in new task
  - Typically 5-10 epochs is sufficient, easy to overfit (very fast)
  - Larger training step size is okay
- **Step 3:** Fine-tune, **unfreeze** a few layers in base model:
  - Attach newly trained model to pre-trained model, *Optional: use augmentation*
  - Train to your hearts content, use smaller training step size



Step 4: Rinse and Repeat unfreezing more layers from base model

```
from torchvision.models import convnext_tiny,
                               ConvNeXt_Tiny_Weights

weights = ConvNeXt_Tiny_Weights.DEFAULT
model = convnext_tiny(weights=weights)

#    (model.features) -> (many CN blocks)
#    (model.avgpool) -> (pools down resolution)
#    (model.classifier) -> ImageNet, replace this
```

Transfer Learning, Bottleneck

```
num_features = 768
num_classes = 200  # Replace with your dataset target count

# Overwrite only the specific linear layer inside the sequential block
model.classifier[1] = nn.Dropout(p=0.5)
model.classifier[2] = nn.Flatten(start_dim=1, end_dim=-1)
model.classifier.append( nn.Linear(num_features, num_classes) ) # new trainable layer!
model = model.to(device)
```

```
# also need to use the correct transforms for this model:
base_transforms = weights.transforms()
train_loader, test_loader, class_names = load_tiny_imagenet(base_transforms)
```

```
from torch.utils.data import TensorDataset

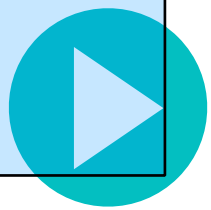
def get_bottleneck(loader, model, shuffle=True):
    features_list = [], labels_list = []

    with torch.no_grad():
        model.eval()
        for images, labels in loader:
            images = images.to(device)
            bottleneck = model.avgpool(model.features(images))
            features_list.append(bottleneck.cpu())
            labels_list.append(labels.cpu())
    all_features = torch.cat(features_list, dim=0)
    all_labels = torch.cat(labels_list, dim=0)

    return DataLoader(TensorDataset(all_features, all_labels),  batch_size=64,
                                                shuffle=shuffle, pin_memory=False )
```

TinyImageNet: 200 classes subset

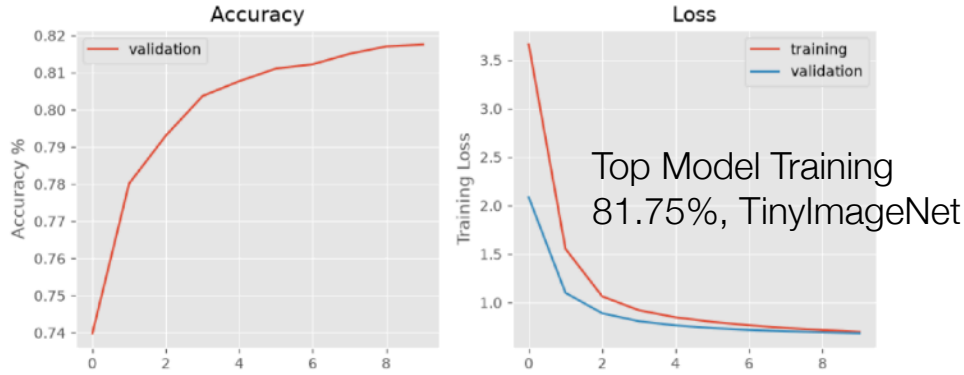
```
200
{'n01443537': 'goldfish,',
 'n01629819': 'European',
 'n01641577': 'bullfrog,',
 'n01644900': 'tailed',
 'n01698640': 'American',
 'n01742172': 'boa',
 'n01768244': 'trilobite',
 'n01770393': 'scorpion',
 'n01774384': 'black',
 'n01774750': 'tarantula',
 'n01784675': 'centipede',
 'n01855672': 'goose',
 'n01882714': 'koala,',
 'n01910747': 'jellyfish',
 'n01917289': 'brain',
 'n01944390': 'snail',
 'n01945685': 'slug',
 'n01950731': 'sea',
 'n01983481': 'American',
 'n01984695': 'spiny',
 'n02002724': 'black',
 'n02056570': 'king',
 'n02058221': 'albatross,',
 'n02074367': 'dugong,
```



Transfer Learning, Make Models

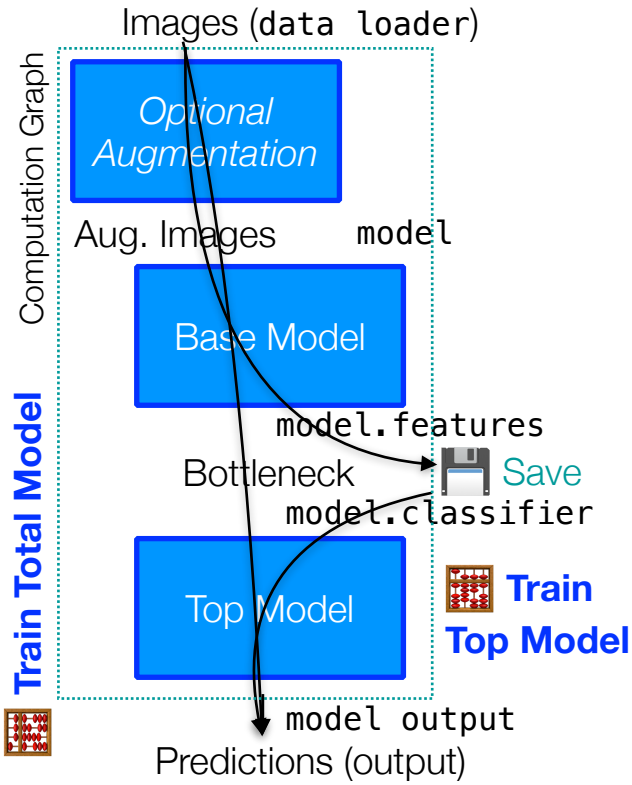
```
optimizer = optim.Adam(model.classifier.parameters(),
                       lr=1e-4)

train_model(model.classifier, … bottleneck loaders …,
    nn.CrossEntropyLoss(), optimizer, epochs=10)
```



Top Model Training
81.75%, TinyImageNet

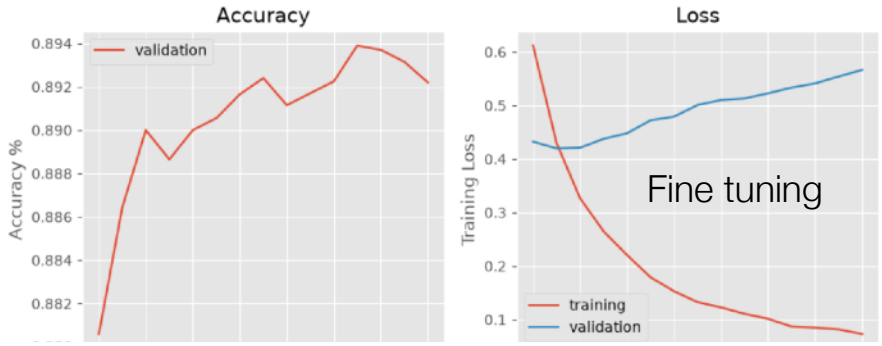
…Then Fit top model with bottleneck features…




```
optimizer = optim.AdamW([
    {"params": model.features.parameters(), "lr": 1e-5}
    {"params": model.classifier.parameters(), "lr": 1e-3}
],)

train_model(model, … image loaders …,
    nn.CrossEntropyLoss(), optimizer, epochs=15)
```

… Fit full model (top and Base) with images…

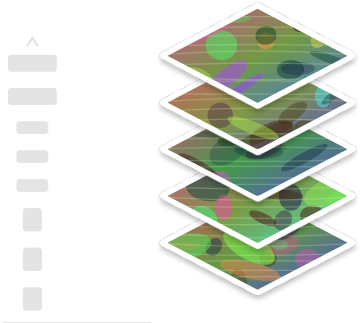


Full Model Tuning
89.2%, TinyImageNet

Many Pre-trained Models to choose from!

AlexNet

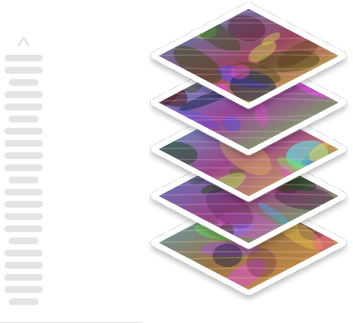
The 2012 ImageNet winner that launched deep learning in vision: five conv layers, ReLU, dropout.



2012 · ~61M params

VGG 19

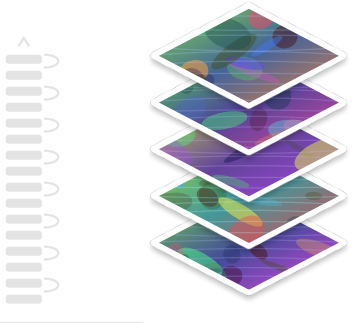
Stacks of plain 3x3 convolutions, 19 layers deep and no branches. Showed that depth alone helps.



2014 · ~144M params

ResNet 50

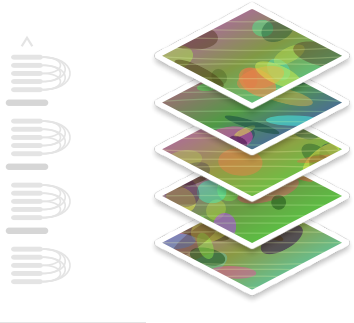
Skip connections let gradients flow through 50 layers, making much deeper networks trainable.



2015 · ~26M params

DenseNet 121

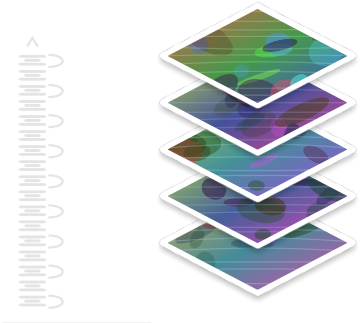
Each layer receives the feature maps of all earlier layers in its block, reusing features cheaply.



2017 · ~8M params

MobileNet v2

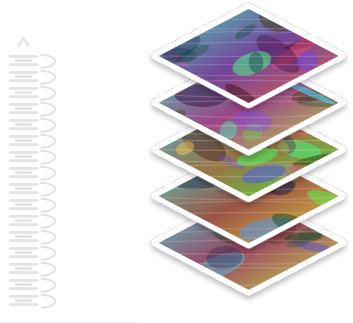
Inverted residual blocks and depthwise convolutions, built for mobile devices.



2018 · ~3.5M params

EfficientNet B0

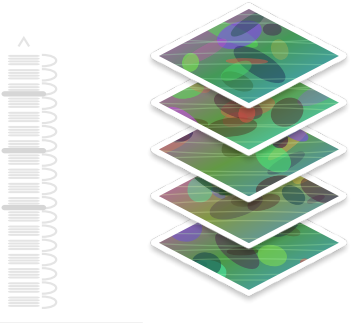
Found by architecture search, then scaled by depth, width, and resolution together.



2019 · ~5M params

ConvNeXt Tiny

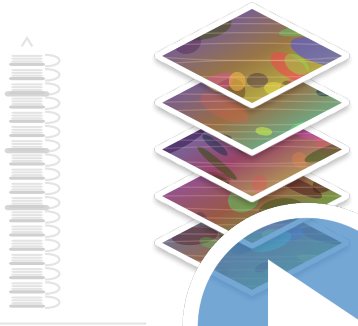
A ResNet modernized with Transformer-era choices: 7x7 depthwise convs, LayerNorm, GELU.



2022 · ~29M params

ConvNeXt V2

Adds masked-autoencoder pretraining and Global Response Normalization.



2023 · ~29M params

More Examples: 10. PyTorch More Advanced CNN Techniques.ipynb



CNN Town Hall

Thanks to Machine Learning the **robot apocalypse** was short lived!

History of Convolutional Neural Networks



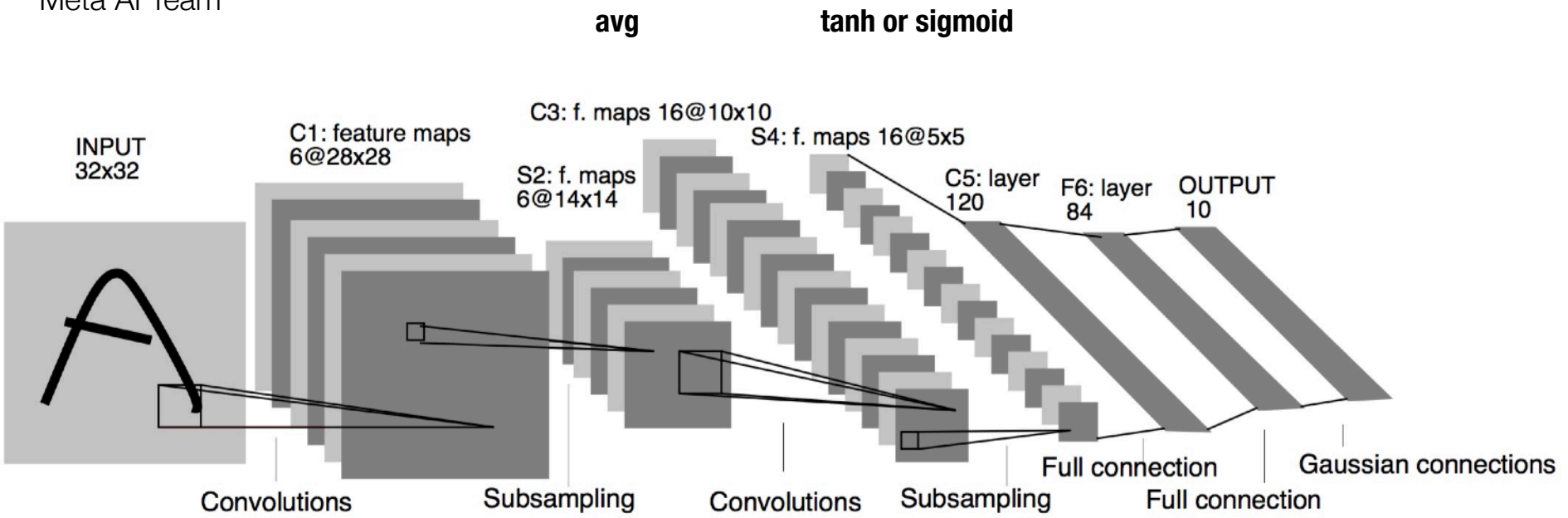
Machine Learning 101

Types of CNN, 1988-1998



Former Lead of Meta AI Team

- **LeNet-1** (1988)
  - ~2600 params, not many layers
- **LeNet-5** (1998)
  - 7 layers, gets excellent MNIST performance
- Major contribution, general structure:
  - conv=>pool=>non-linearity=> …=>MLP



CNN History

- List of major breakthroughs from 1998 through 2010 for convolutional networks:

¯_(ツ)_/¯

- 2010



Types of CNN, 2010



AI Researcher
IDSA, Switzerland

- **Ciresan Net**
- Publishes code for running CNN via GPU
  - Subsequently wins 5 international competitions
  - from stop signs => cancer detection
- Major contribution: NVIDIA parallelized training algorithms

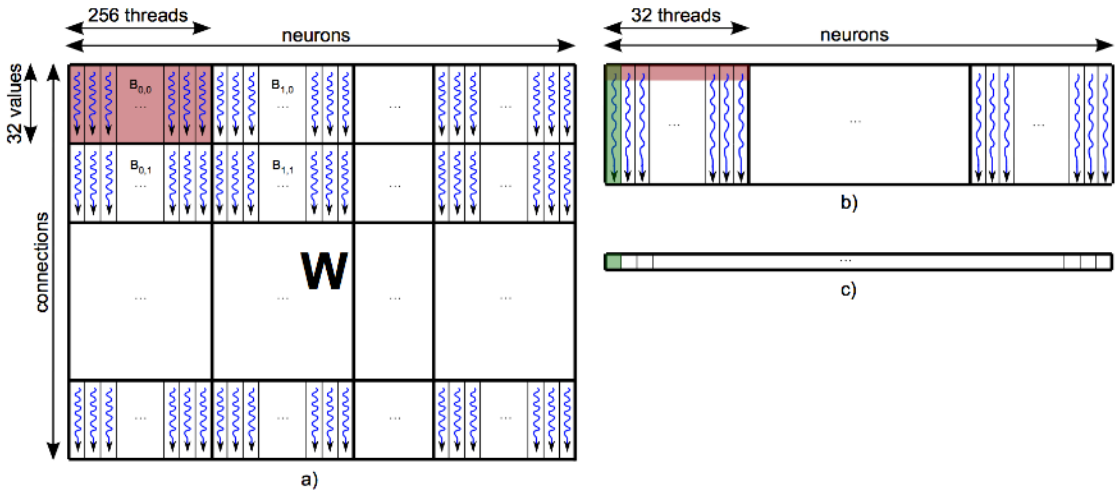


Figure 2: Forward propagation: a) mapping of kernel 1 grid onto the padded weight matrix; b) mapping the kernel 2 grid onto the partial dot products matrix; c) output of forward propagation.

https://arxiv.org/pdf/1003.0358.pdf

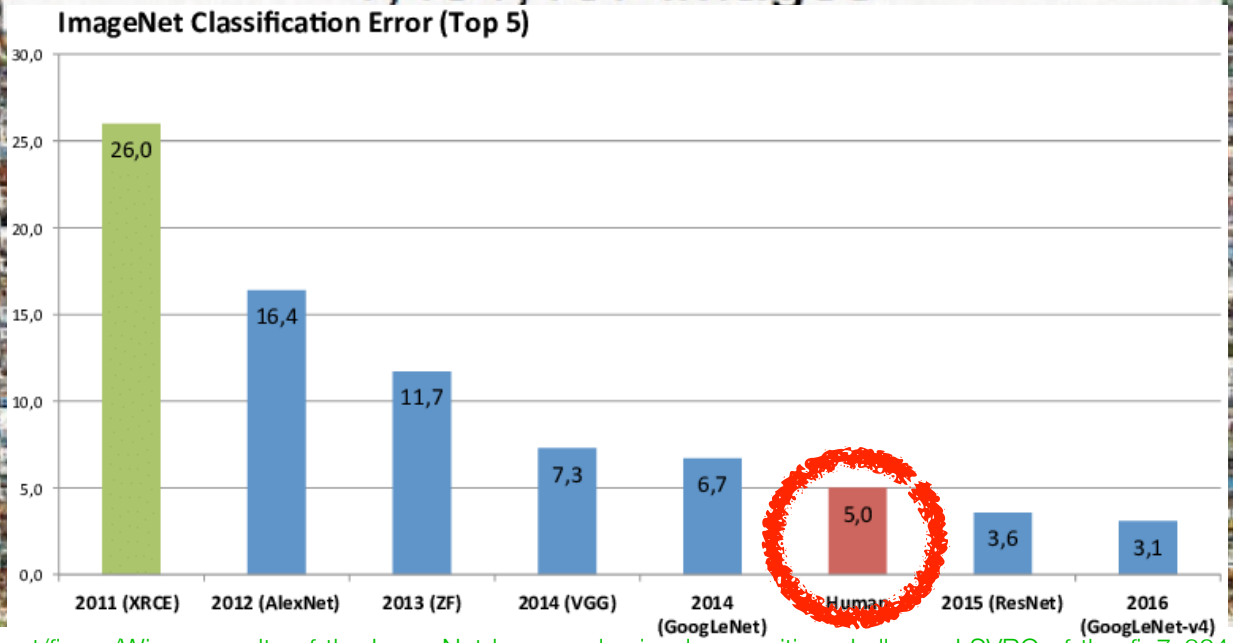
ImageNet Competition (2010-2016)

IMAGENET Large Scale Visual Recognition Challenge

The Image Classification Challenge:

1,000 object classes

1,431,167 images



https://www.researchgate.net/figure/Winner-results-of-the-ImageNet-large-scale-visual-recognition-challenge-LSVRC-of-the_fig7_324476862

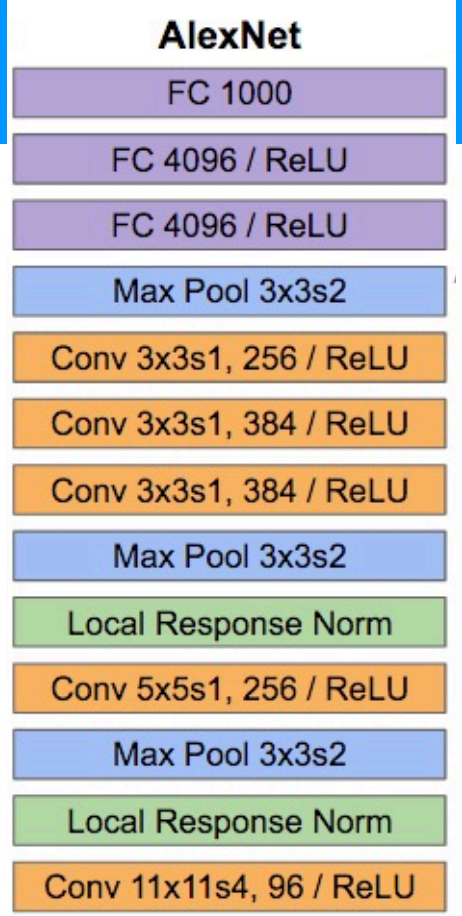
https://www.slideshare.net/nmhkahn/case-study-of-convolutional-neural-network-61556303

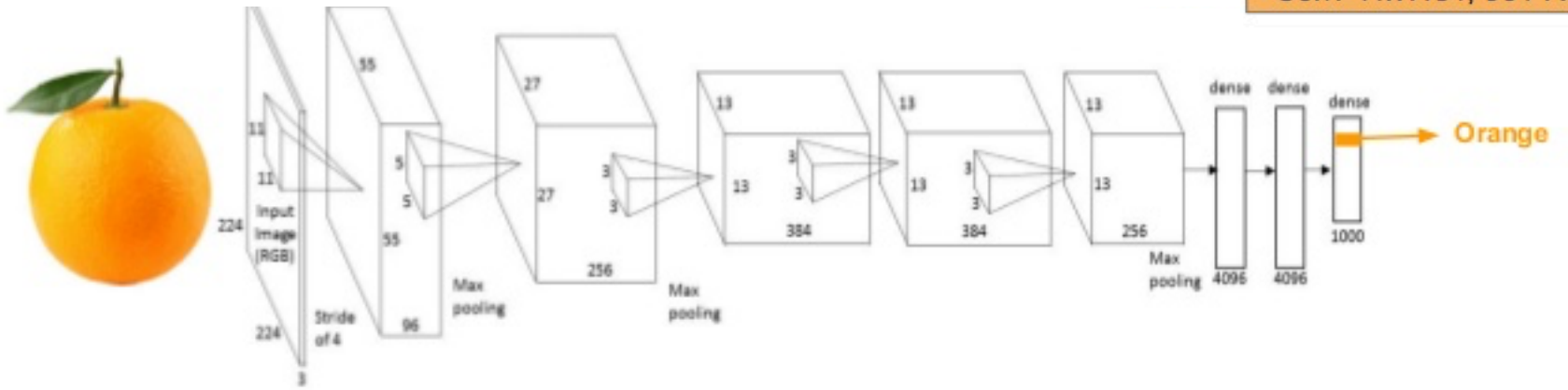
Types of CNN, 2012



Google

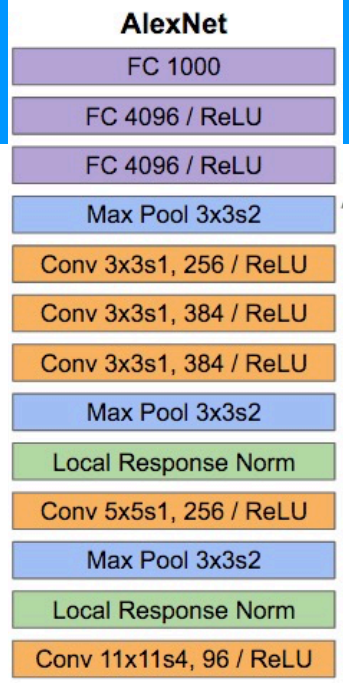
- **AlexNet**, Hinton is mentor
  - wins ImageNet competition
- Major contributions:
  - dropout for regularization
  - systematic use of ReLU
  - data expansion
  - ***overlapping max pool***





https://culurciello.github.io/tech/2016/06/04/nets.html

AlexNet, in Code




```python
class TinyAlexNet(nn.Module):
    def __init__(self, num_classes: int = 200, dropout: float = 0.5):
        super().__init__()
        self.name = 'TinyAlexnet'
        self.features = nn.Sequential(
            #                          strided conv     max pool
            # Conv 1: (3, 64, 64) -> (32, 32, 32) -> (32, 16, 16)
            nn.Conv2d(3, 32, kernel_size=5, stride=2, padding=2),
            LayerNorm2d(32),
            nn.GELU(approximate='tanh'),
            nn.MaxPool2d(kernel_size=2, stride=2),

            # Conv 2: (32, 16, 16) -> (64, 16, 16) -> (64, 8, 8)
            nn.Conv2d(32, 64, kernel_size=5, padding=2),
            LayerNorm2d(64),
            nn.GELU(approximate='tanh'),
            nn.MaxPool2d(kernel_size=2, stride=2),

            # Conv 3: (64, 8, 8) -> (128, 8, 8) > (128, 4, 4)
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            LayerNorm2d(128),
            nn.GELU(approximate='tanh'),
            nn.MaxPool2d(kernel_size=2, stride=2),

        )

        # Spatial resolution: 128 * 4 * 4 = 2048 features
        self.classifier = nn.Sequential(
            nn.Dropout(p=dropout),
            nn.Linear(128 * 4 * 4, 1024),
            nn.LayerNorm(1024),
            nn.ReLU(inplace=True),
            nn.Dropout(p=dropout),
            nn.Linear(1024, 512),
            nn.LayerNorm(512),
            nn.ReLU(inplace=True),
            nn.Linear(512, num_classes),
        )
```

...

```python
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x

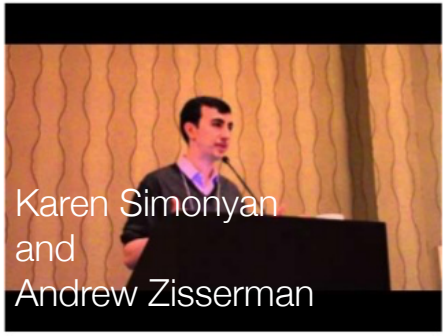
alexnet = TinyAlexNet().to(device)
alexnet
```

10. PyTorch More Advanced CNN Techniques.ipynb

Warning


DON'T TRY WHAT YOU'RE ABOUT TO SEE AT HOME
WE'RE WHAT YOU CALL EXPERTS
WeKnowMemes

Types of CNN, 2013



UNIVERSITY OF OXFORD

Google DeepMind

- Oxford **VGG Net** (Visual Geometry Group)
- Major contributions:
  - small cascaded kernels
  - way more layers (19 versus ~7)
  - *"emulates" biology "better"* 😒
  - trained on NVIDIA GPUs for 2-3 weeks

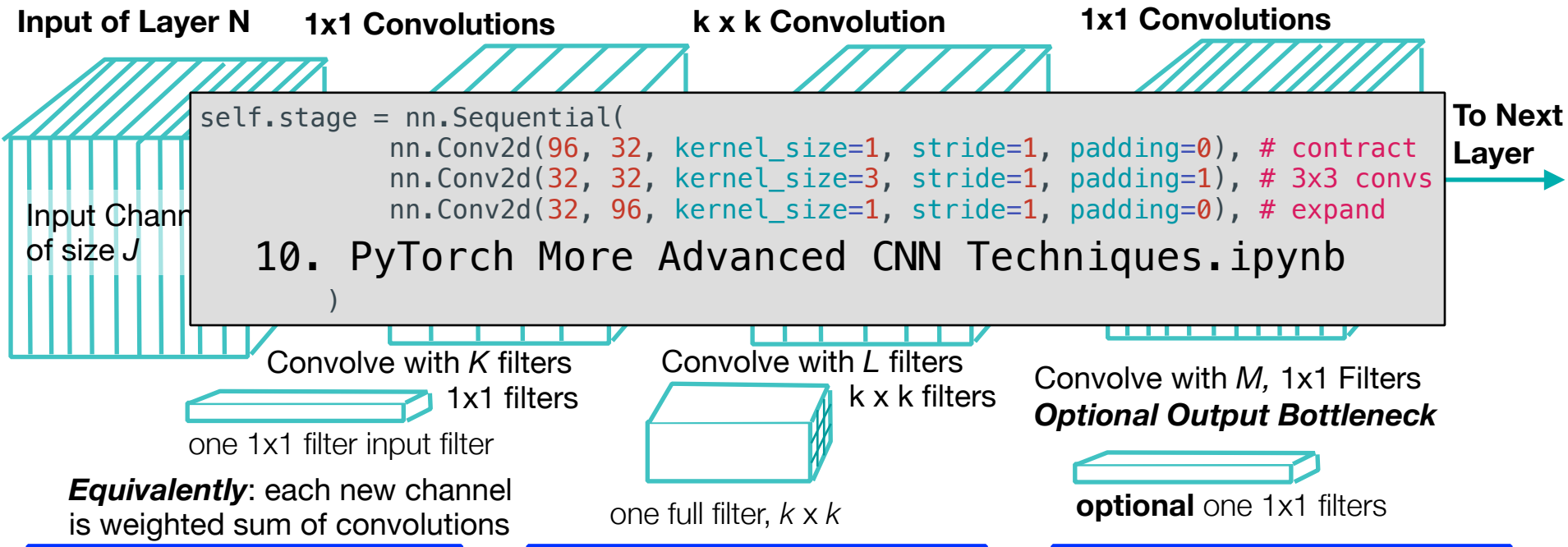
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

http://www.robots.ox.ac.uk/~vgg/practicals/cnn/

2014 Network in Network, expanded view



Reduce Channels using Lightweight 1x1 Filters

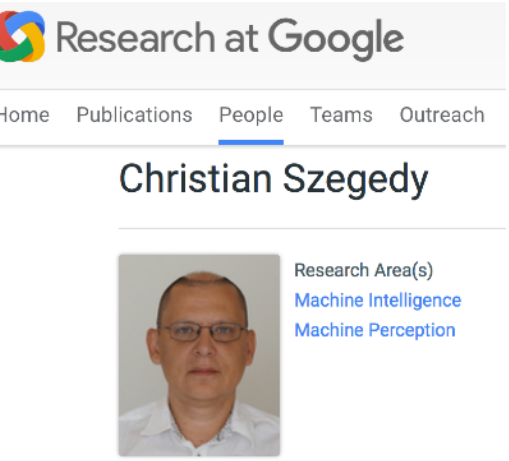
Larger k x k Filters on Reduced Channels

Expand with lightweight Filters, *(if desired)*

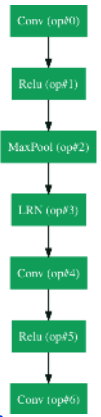
Example: Train 32 filters with kernel size of 3x3, input channels of 256 (outputs size of 32)

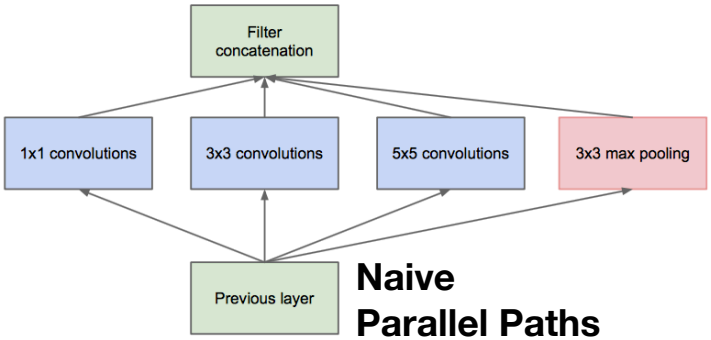
- No reduction: (32 x 3 x 3 x 256) = **73,728** (no bottleneck)
- Reduction to 128: (128 x 1 x 1 x 256) + (32 x 3 x 3 x 128) = **69,632** (reduce 128)
- Reduction to 64: (64 x 1 x 1 x 256) + (32 x 3 x 3 x 64) = **34,816** (reduce 64)

Types of CNN, 2014 (parallel pathways)

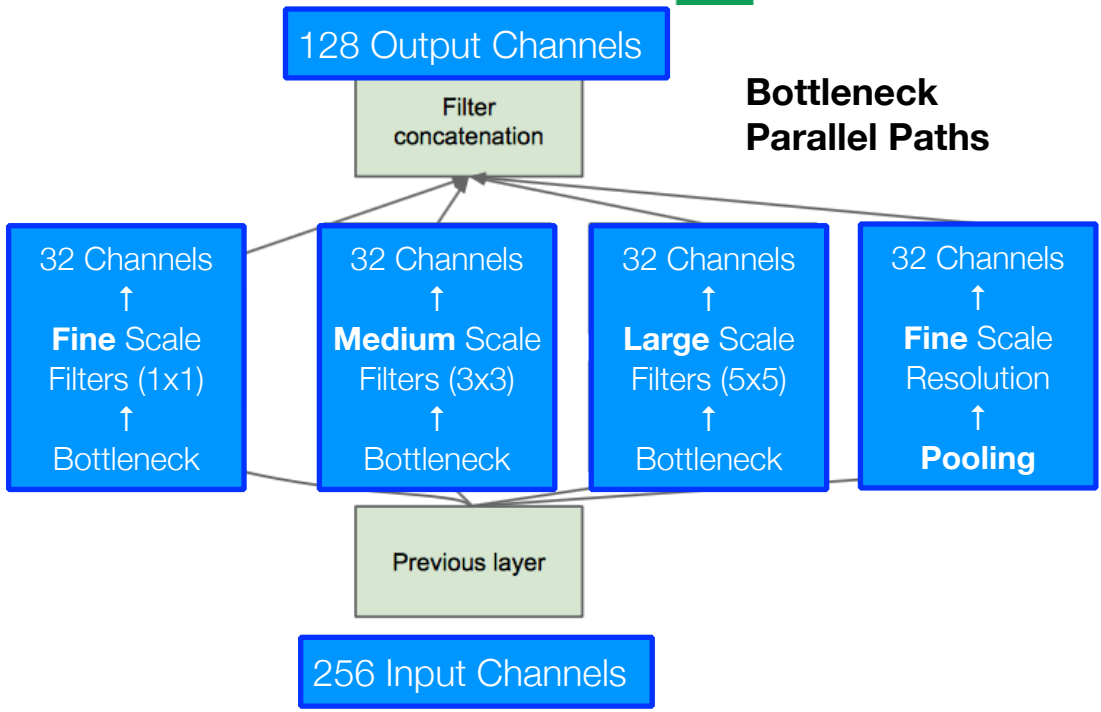


- **GoogLeNet**
  - or **Inception V1**
- Major contribution:
  - multi-scale filtering
  - parallel NiN





Naive Parallel Paths



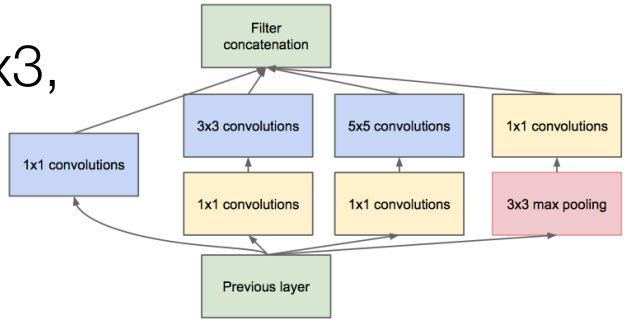
Bottleneck Parallel Paths

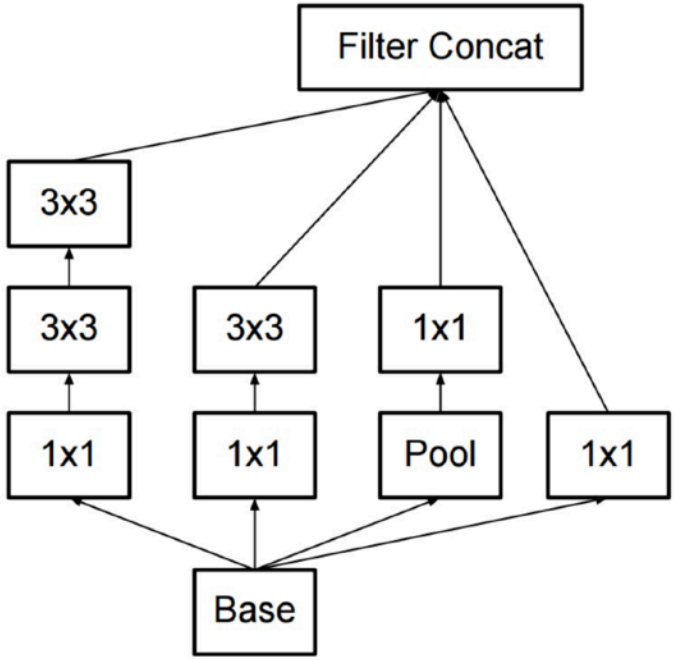
https://arxiv.org/pdf/1409.4842.pdf

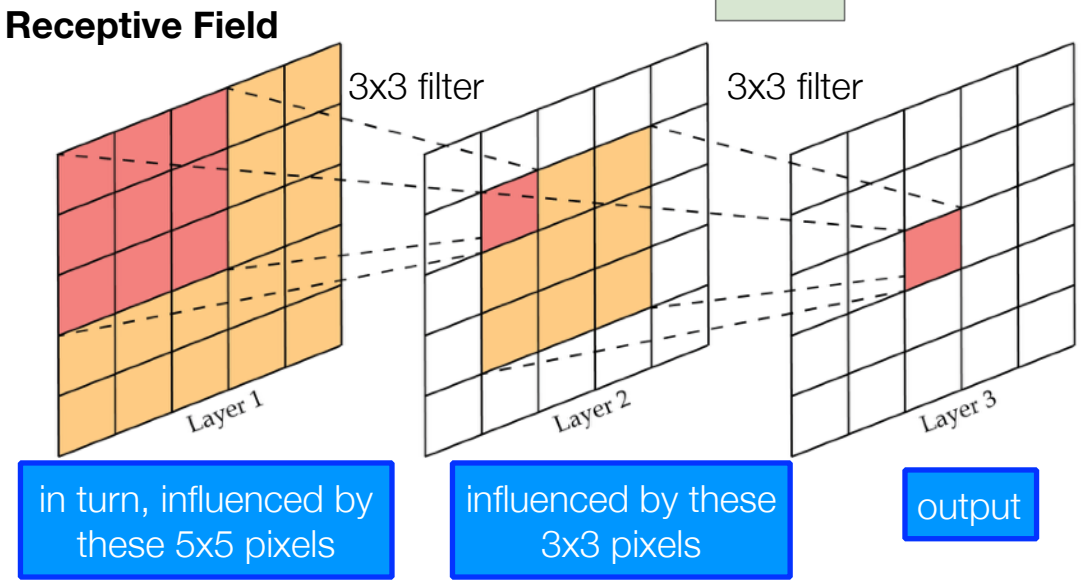
Types of CNN, 2015 February and December



- **Inception V2**,
  - Inception V1 with different normalization
- **Inception V3**:
  - replace 5x5 with multiple 3x3, maintains receptive field







https://arxiv.org/pdf/1512.00567.pdf

https://medium.com/@rekalantar/receptive-fields-in-deep-convolutional-networks-43871d2ef2e9